
WEB-PDE-LLM: A Multi-Agent Framework for Real-Time Web-Based PDE Solving

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Partial differential equation (PDE) simulation is central to scientific computing,
2 but building usable solvers still requires relevant expertise and substantial local
3 software infrastructure. We present WEB-PDE-LLM, a multi-agent framework
4 that translates mathematical PDE descriptions into browser-based WebGL solvers:
5 specialized agents parse the PDE specification, validate numerical metadata, verify
6 execution in a browser, and repair solver code using compilation, and numerical
7 errors feedback. Unlike prior LLM-based simulation pipelines that generate Python
8 solvers, WEB-PDE-LLM targets the shader programming layer itself, enabling
9 real-time, interactive PDE simulation on consumer devices without a Python and
10 GPU setup. We evaluate on five PDE families ranging from one-dimensional
11 advection to cardiac electrophysiology across four LLM backbones, comparing
12 against one-shot LLM generation and state-of-the-art methods (CodePDE, PINN-
13 Agent, MCP-SIM) on test nRMSE, pass rate, and token cost. WEB-PDE-LLM
14 is the only compared method that produces a valid solver on the hardest cardiac
15 model across all backbones, and attains the highest accuracy and pass-rate scores
16 overall.

17 1 Introduction

18 PDEs govern transport, diffusion, wave propagation, and many biological and physical processes.
19 In cardiac electrophysiology, reaction-diffusion PDEs couple membrane-voltage propagation with
20 nonlinear ionic kinetics, producing wave fronts, spiral waves, and fibrillatory dynamics that are
21 costly to simulate at high resolution [1, 2]. Even with mature numerical methods, the workflow
22 is demanding: discretization, boundary conditions, stable time steps, and a compute environment.
23 LLMs have begun to lower it by generating scientific code from natural language, part of a broader
24 shift toward agentic workflows that combine reasoning, tool use, execution feedback, and iterative
25 self-correction [3–6]. Recent systems such as CodePDE and MCP-SIM show that LLMs can generate,
26 debug, and refine simulation code [7, 8], but they still assume a conventional execution substrate such
27 as Python or domain-specific configurations, so even a correct solver needs a compatible environment
28 and local hardware. However, none of these methods target browser-based simulation, where physical
29 fields are maintained as GPU textures and numerical updates are executed via fragment shaders
30 [9, 10].

31 WEB-PDE-LLM closes this gap by treating the browser itself as the simulation platform, converting
32 a LaTeX-like PDE description into a WebGL application whose fragment shader advances the state
33 in real time, a self-contained simulation that needs no Python, or local hardware setup. Because
34 encoding PDEs directly into WebGL textures is non-trivial, WEB-PDE-LLM adopts a multi-agent
35 framework (Section 3) that decouples equation parsing, code generation, verification, and automated
36 debugging [11, 12].

37 The main contributions of this work are:

- 38 • **First LLM-to-WebGL Framework:** We design the first automated pipeline translating
39 mathematical PDE specifications into browser-executable WebGL solvers.
- 40 • **Zero-Setup In-Browser Simulation:** We enable direct synthesis of interactive WebGL
41 solvers from natural language, executing client-side on personal hardware without depen-
42 dencies on Python runtimes or external numerical libraries.
- 43 • **Specialized Multi-Agent Workflow:** We introduce a modular architecture tailored to shader
44 synthesis that decouples equation parsing, solver generation, headless browser verification,
45 and automated debugging.
- 46 • **Rigorous Benchmark on Complex Dynamics:** We evaluate across five PDE families
47 with an emphasis on multiscale cardiac electrophysiology, systematically benchmarking
48 against one-shot LLM generation and state-of-the-art frameworks (CodePDE, MCP-SIM,
49 PINNsAgent) across numerical accuracy, pass rate, and token efficiency.

50 2 Related Work

51 **LLM-based PDE solver generation.** CodePDE is the closest baseline in spirit, asking LLMs to
52 generate, execute, debug, and refine PDE solvers at inference time [7]; WEB-PDE-LLM shares this
53 code-as-reasoning premise but targets WebGL2 fragment shaders and browser applications, changing
54 the failure modes and required abstractions. OpInf-LLM instead learns a data-driven surrogate offline
55 via operator inference, emitting no inspectable solver [13]. PDEBench benchmarks classical and
56 learned solvers [14], and physics-informed neural networks and neural operators learn solution maps
57 from data or physics constraints [15, 16].

58 **Multi-agent systems for scientific simulation.** Multi-agent LLM systems decompose complex
59 tasks into specialized planning, coding, execution, and repair roles [6, 17–21]. MCP-SIM builds, exe-
60 cutes, and diagnoses simulation code via a memory-coordinated agent set [8]; CFDAgent automates
61 zero-shot CFD simulation with preprocessing, solver, and postprocessing agents [22]; related work
62 applies similar coordination to scientific hypothesis generation [23].

63 3 Methodology

64 3.1 Problem formulation

65 The input to WEB-PDE-LLM is a LaTeX-like description of a time-dependent PDE system,
66 $\partial_t \mathbf{q}(t, \mathbf{x}) = \mathcal{F}(\mathbf{q}, \nabla \mathbf{q}, \nabla^2 \mathbf{q}; \boldsymbol{\theta})$, where $\mathbf{q}$ denotes one or more state variables and $\boldsymbol{\theta}$ physical/nu-
67 merical parameters, together with the spatial domain, grid resolution, time horizon, time step, and
68 boundary conditions. The output is a WebGL simulation artifact that advances $\mathbf{q}$ on a texture grid
69 and, when requested for evaluation, exports sampled solution values for comparison with reference
70 data. The implementation covers one-dimensional advection and Burgers’, two-dimensional heat,
71 Fenton–Karma (FK), and Ten Tusscher–Noble–Noble–Panfilov (TNNP) [1, 2]: the simpler PDEs test
72 stable transport/diffusion stencils, while the cardiac models test coupled nonlinear reaction terms,
73 piecewise kinetics, and multi-channel state layouts.

74 3.2 System overview

75 WEB-PDE-LLM is organized as a parse/code/verify/debug loop (Figure 1), following the agentic
76 pattern of interleaving reasoning, action, external execution, and feedback-driven revision [3–5]. The
77 framework is model-provider agnostic and routes through OpenAI, Anthropic, Gemini, DeepSeek, or
78 cloud/local Ollama models. Parse validation matters most for smaller models, which may hallucinate
79 fields or choose unstable time steps, motivating explicit checker and resolver roles [11, 21].

80 3.3 Agent details

81 The **Parse Agent** converts the description into a JSON object with the fields needed downstream:
82 governing equations, state-variable count, texture size, spatial and time step, domain size, time

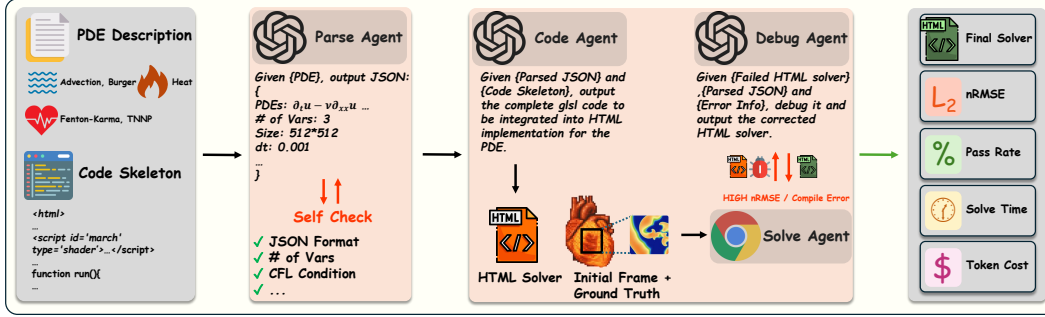


Figure 1: The WEB-PDE-LLM pipeline. A PDE description is parsed into structured metadata, checked, injected into a WebGL2/GLSL skeleton, completed by the Code Agent, and run in a browser by the Verify Agent. Compilation, runtime, and numerical-error feedback returns to the Debug Agent until the shader matches the reference trajectory.

horizon, boundary conditions, parameters, and implementation notes. A **parsed-content checking agent** then reviews it for completeness, type correctness, and numerical plausibility (especially CFL-like time-step stability), correcting unstable metadata [5, 11, 6]. Based on the state-variable count, **Skeleton Preparation** selects a WebGL texture layout (a single channel for scalar PDEs, three channels for the FK, twenty channels for the TNNP) and injects Δt , Δx , texture size, and parameters. This narrows the **Code Agent** to the mathematically critical kernel: raw GLSL implementing finite-difference Laplacians, upwind updates, and thresholded gating kinetics for the cardiac models [12]. The **Verify Agent** launches the browser simulation, monitors WebGL and console logs, and computes numerical error. If then execution fails or error is too high, it triggers an iterative debugging loop (up to 5 attempts) where the **Debug Agent** diagnoses the feedback and refines the shader code."

4 Experiments

4.1 PDEs of Interest

Our benchmark suite comprises five PDE families, chosen to stress spatial dimension, operator type (transport, diffusion, mixed), linearity, state-variable count (one to twenty), and boundary conditions (periodic, no-flux/Neumann). For every case the generated solver advances the state from a prescribed initial condition to a fixed horizon T and exports sampled trajectories for comparison against reference solutions; full discretization and parameter values are released with our code.

Advection. The one-dimensional linear advection equation $\partial_t u = -\beta \partial_x u$, $x \in (0, 1)$, $t \in (0, T]$, with periodic boundaries and a smooth initial condition, tests stable transport stencils on a 1024-point grid to $T = 2$. β is chosen to be 0.1.

Burgers'. The one-dimensional viscous Burgers' equation in conservative form, $\partial_t u + \partial_x (\frac{1}{2} u^2) = \frac{\nu}{\pi} \partial_{xx} u$, $x \in (0, 1)$ with periodic boundary condition, 1024-point grid and $T = 2$. ν is chosen to be 0.001.

Heat. The two-dimensional heat equation $\partial_t u = \alpha \nabla^2 u$, $(x, y) \in (0, 1)^2$, with periodic boundaries and diffusivity α , tests an isotropic diffusion stencil on a 128×128 grid to $T \approx 45.8$. α is chosen to be 0.01

FK. The FK model is a two-dimensional, three-variable reaction-diffusion system for a diffusive field $u(t, \mathbf{x})$ and non-diffusive gating variables v, w , coupled through three piecewise ionic currents gated by a Heaviside excitation threshold (full equations in Section A). We solve on $\Omega = [0, 10]^2$ with no-flux (Neumann) boundaries on a 512×512 grid, $T = 100$, $\Delta t = 2.5 \times 10^{-2}$, $D = 10^{-3}$, using the standard FK parameter set [1].

TNNP. The TNNP model couples a diffusive transmembrane potential $V(t, \mathbf{x})$ to nineteen non-diffusive state variables through a total ionic current that sums twelve nonlinear components (complete

Table 1: Qualitative evaluation of generated solver capabilities.

Method	Cross-Platform Compatibility	Zero- Configuration	Real-Time Visualization	Autonomous Debugging	No Training Required
WEB-PDE-LLM	✓	✓	✓	✓	✓
CodePDE	✓	✗	✗	✓	✓
LLM-only	✓	✗	✗	✗	✓
MCP-SIM	✗	✗	✗	✓ ^a	✓
PINNsAgent	✓	✗	✗	✗	✗
Op-Inf LLM	✓	✗	✗	✓	✗

^a MCP-SIM resolves system runtime bugs but cannot correct numerical accuracy issues (e.g., high nRMSE).

equations and standard TP06 parameters in Section A). We integrate on $\Omega = [0, 12]^2$ with no-flux (Neumann) boundaries on a 512×512 grid, $T = 100$, $\Delta t = 2.5 \times 10^{-2}$, $D = 10^{-3}$.

4.2 Results

We evaluate every framework on the five PDE families of Section 4.1 across four LLM backbones (Gemini 3.5 Flash, GPT-5.5, Claude Opus 4.8, and DeepSeek V4-Pro) along three metrics: accuracy (normalized Root Mean Square Error, nRMSE), pass rate, and API token cost. Throughout the result tables, the best value in each PDE column is shaded gray and the second best is underlined. The main text reports accuracy. The full pass rate, token cost and ablation experiment results are deferred to Section B.

Table 1 compares WEB-PDE-LLM qualitatively against one-shot generation, CodePDE-style solver synthesis, and multi-agent simulation frameworks: it is the only method that is simultaneously cross-platform, zero-configuration, real-time-visualized, autonomously debugged, and requires no training [7, 8, 22, 19, 18].

Cell-wise Score. Because several methods failed on the harder PDEs, a geometric mean over only the solved cells would reward a method for reporting results on easy problems while omitting its failures. We therefore summarize each method with a coverage-aware score that grades every PDE relative to the five competing methods (WEB-PDE-LLM, LLM-only, CodePDE, MCP-Sim, PINNsAgent) so a method cannot score well by solving only the easy PDEs. Per-PDE accuracy is the normalized RMSE, $\text{nRMSE} = \frac{1}{S} \sum_{s=1}^S \|\mathbf{u}^{(s)} - \hat{\mathbf{u}}^{(s)}\|_2 / \|\mathbf{u}^{(s)}\|_2$, averaged over the S evaluated snapshots of that PDE. Token cost is graded the same way. Pass rate enters directly on its raw 0–100 scale (higher is better); exact formulas are in Section A.3.

Overall comparison. Figure 2 summarizes the three metrics under this Score. WEB-PDE-LLM leads on accuracy (87.3) and pass rate (90.5), ahead of CodePDE (51.3, 78.5) and well ahead of MCP-Sim (19.2, 47.5). The gap comes mostly from coverage on the cardiac models, where the Python baselines often fail to produce a running solver. Its weakest metric is token efficiency (45.6): the parse-code-verify-debug loop calls LLM several times, though it still beats the MCP-Sim loop, the most token-hungry method on every PDE (0.9). One-shot LLM-only generation is the cheapest in tokens (87.4). PINNsAgent is close behind (77.2) but least accurate (6.7), trading LLM calls for network training.

Accuracy. Table 2 reports test nRMSE per PDE and backbone. On the cardiac models the separation is categorical. WEB-PDE-LLM is the only framework that produces a valid TNNP solver on any backbone, down to 4.67×10^{-6} on GPT-5.5, while every Python baseline reports “–”. On FK its best run reaches 2.80×10^{-4} , on par with the strongest CodePDE run (2.79×10^{-4}), while MCP-Sim never completes an FK case. On the 1D cases the shader solvers are about an order of magnitude more accurate than the Python baselines, which we attribute to the parsed-metadata stage selecting a stable Δt and an upwind-consistent stencil. Heat is the exception: the best Python baselines reach the 10^{-7} range while WEB-PDE-LLM stays near 10^{-4} , at the single-precision-texture floor. PINNsAgent trails every code-generating method by two to four orders of magnitude. Figure 3 in Section B.2 shows the predicted fields and pointwise errors behind these numbers for the Gemini 3.5 Flash backbone.

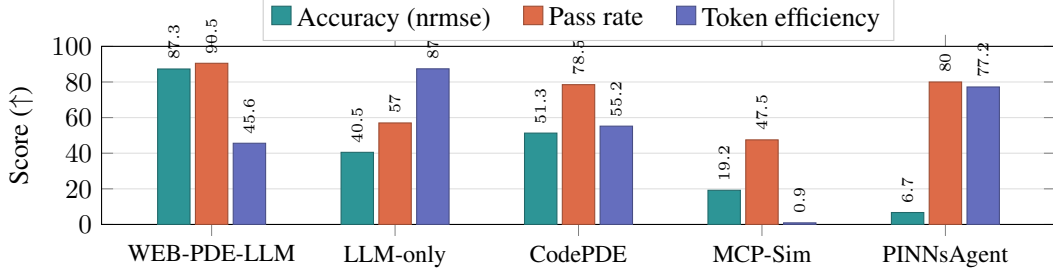


Figure 2: Coverage-aware *Score* per metric (higher is better, 0–100), averaged over four LLM backbones (Gemini 3.5 Flash, GPT-5.5, Claude Opus 4.8, DeepSeek V4-Pro). Each metric is graded per PDE relative to the competing frameworks (best → 100, worst → 0, missing → 0) and averaged over the five PDEs; nRMSE and token cost use log-relative grading, pass rate is scored directly on its 0–100 scale (all: higher score is better).

Reliability and cost. Per-PDE pass-rate and token-cost measurements are tabulated in Section B. WEB-PDE-LLM has the strongest TNNP pass rate (50–100%) and stays at 70–90% on FK; CodePDE is reliable on simpler cases but never produces a running TNNP solver, and MCP-Sim degrades earliest, dropping to 40–80% on Burgers (its only TNNP success: one DeepSeek V4-Pro scale, 10%). The multi-agent loop costs 2–4× one-shot tokens on simple cases, yet is cheaper end-to-end (\$1.39–\$13.76/backbone) than CodePDE (\$2.15–\$21.64) or MCP-Sim (\$2.49–\$31.54), both burning roughly half a million TNNP tokens per backbone without a working solver.

Ablations. To isolate WEB-PDE-LLM’s two agents, we remove them cumulatively on GPT-5.5: dropping parse-validation degrades accuracy on four of five PDEs and drops mean pass rate 96.0% → 84.0%; further removing the Debug Agent costs another order of magnitude on advection and 4 more points (Section B.3).

5 Conclusion

We introduced WEB-PDE-LLM, a multi-agent framework that compiles natural-language PDE descriptions into browser-native WebGL2/GLSL simulations, together with an evaluation protocol spanning five PDE families and four LLM backbones. WEB-PDE-LLM outperforms one-shot generation, CodePDE-style Python synthesis, multi-agent simulation, and trained PINN surrogates on accuracy, reliability, and cost, most markedly on the two cardiac models, where every Python baseline fails to produce a running TNNP solver. **Limitations and future work.** The benchmark covers five PDE families on regular grids with periodic or no-flux boundaries; irregular geometries, adaptive meshing, and 3D domains fall outside the current WebGL2 texture representation. Heat-equation accuracy is capped by single-precision textures, a hardware limit some WebGL2 extensions could partially relax. Each PINNsAgent result also searches on a single held-out instance under a fixed prompt budget; a wider instance sweep, adaptive budgets, and extending the shader-generation loop to larger PDE systems are natural next steps.

References

- [1] Flavio H. Fenton and Alain Karma. Vortex dynamics in three-dimensional continuous myocardium with fiber rotation: Filament instability and fibrillation. *Chaos*, 8(1):20–47, 1998.
- [2] K. H. W. J. ten Tusscher, D. Noble, P. J. Noble, and A. V. Panfilov. A model for human ventricular tissue. *American Journal of Physiology-Heart and Circulatory Physiology*, 286(4):H1573–H1589, 2004.
- [3] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. Re-Act: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

- [4] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [6] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Adam White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [7] Shanda Li, Tanya Marwah, Junhong Shen, Weiwei Sun, Andrej Risteski, Yiming Yang, and Ameet Talwalkar. CodePDE: An inference framework for LLM-driven PDE solver generation. *arXiv preprint arXiv:2505.08783*, 2025.
- [8] Donggeun Park, Hyeonbin Moon, and Seunghwa Ryu. A self-correcting multi-agent LLM framework for language-based physics simulation and explanation. *npj Artificial Intelligence*, 2:10, 2026.
- [9] Khronos Group. WebGL 2.0 specification, 2017.
- [10] Khronos Group. OpenGL ES Shading Language 3.00.6 specification, 2016. Accessed 2026-05-04.
- [11] Varun Nair, Elliot Schumacher, Geoffrey Tso, and Anitha Kannan. DERA: Enhancing large language model completions with dialog-enabled resolving agents. In *Proceedings of the 6th Clinical Natural Language Processing Workshop*, 2024.
- [12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [13] Zhuoyuan Wang, Hanjiang Hu, Xiyu Deng, Saviz Mowlavi, and Yorie Nakahira. OpInf-LLM: Parametric PDE solving with LLMs via operator inference. *arXiv preprint arXiv:2602.01493*, 2026.
- [14] Makoto Takamoto, Timothy Praditia, Raphael Leiteritz, Daniel MacKinlay, Francesco Alesiani, Dirk Pflueger, and Mathias Niepert. PDEBench: An extensive benchmark for scientific machine learning. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [15] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [16] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.
- [17] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [18] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [19] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [20] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. *arXiv preprint arXiv:2308.10848*, 2023.
- [21] Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O’Sullivan, and Hoang D. Nguyen. Multi-agent collaboration mechanisms: A survey of LLMs. *arXiv preprint arXiv:2501.06322*, 2025.

- 239 [22] Zhaoyue Xu, Long Wang, Chunyu Wang, Yixin Chen, Qingyong Luo, Hua-Dong Yao, Shizhao Wang, and
240 Guowei He. CFDAgent: A language-guided, zero-shot multi-agent system for complex flow simulation.
241 *Physics of Fluids*, 37(11):117124, 2025.
- 242 [23] Alireza Ghafarollahi and Markus J. Buehler. SciAgents: Automating scientific discovery through multi-
243 agent intelligent graph reasoning. *arXiv preprint arXiv:2409.05556*, 2024.

244 A Full PDE Definitions and Score Formula

245 This appendix gives the complete state equations for the two cardiac models (Section 4.1) and the
246 exact coverage-aware Score formulas summarized in the *Cell-wise Score* paragraph of the main text.

247 A.1 Fenton–Karma

248 The Fenton–Karma (FK) model is a two-dimensional, three-variable nonlinear reaction-diffusion
249 system for a diffusive field $u(t, \mathbf{x})$ and two non-diffusive gating variables $v(t, \mathbf{x})$ and $w(t, \mathbf{x})$, where
250 only u carries the Laplacian term. The model equations are

$$\begin{aligned}\frac{\partial u(\mathbf{x}, t)}{\partial t} &= D \nabla^2 u - \frac{I_{\text{fi}}(u, v) + I_{\text{so}}(u) + I_{\text{si}}(u, v, w)}{C_m} \\ \frac{\partial v(\mathbf{x}, t)}{\partial t} &= \frac{(1 - \mathcal{H}(u - u_c))(1 - v)}{\tau_v^-(u)} - \frac{\mathcal{H}(u - u_c)v}{\tau_v^+} \\ \frac{\partial w(\mathbf{x}, t)}{\partial t} &= \frac{(1 - \mathcal{H}(u - u_c))(1 - w)}{\tau_w^-} - \frac{\mathcal{H}(u - u_c)w}{\tau_w^+},\end{aligned}\tag{1}$$

251 where the three currents are specified as

$$\begin{aligned}I_{\text{fi}}(u, v) &= -v \mathcal{H}(u - u_c)(u - u_c)(1 - u)/\tau_d \\ I_{\text{so}}(u) &= u(1 - \mathcal{H}(u - u_c))/\tau_0 + \mathcal{H}(u - u_c)/\tau_r \\ I_{\text{si}}(u, w) &= -w(1 + \tanh(k(u - u_c^{\text{si}})))/(2\tau_{\text{si}})\end{aligned}\tag{2}$$

252 with $\tau_v^-(u) = (1 - \mathcal{H}(u - u_v))\tau_{v1}^- - \mathcal{H}(u - u_v)\tau_{v2}^-$ and $\mathcal{H}(\cdot)$ the Heaviside function.

253 A.2 Ten Tusscher–Noble–Noble–Panfilov

254 The ten Tusscher–Noble–Noble–Panfilov (TNNP) model couples a diffusive transmembrane potential
255 $V(t, \mathbf{x})$ to nineteen non-diffusive state variables. The potential evolves as

$$\partial_t V = D \nabla^2 V - \frac{1}{C_m} I_{\text{ion}},\tag{3}$$

256 where the total ionic current sums twelve nonlinear components,

$$\begin{aligned}I_{\text{ion}} &= I_{\text{Na}} + I_{\text{K1}} + I_{\text{lo}} + I_{\text{Kr}} + I_{\text{Ks}} + I_{\text{CaL}} \\ &\quad + I_{\text{NaCa}} + I_{\text{NaK}} + I_{\text{pCa}} + I_{\text{pK}} + I_{\text{bNa}} + I_{\text{bCa}}.\end{aligned}\tag{4}$$

257 Writing $\phi = F/(RT)$, the Nernst reversal potentials are

$$\begin{aligned}E_K &= \frac{1}{\phi} \ln \frac{K_o}{K_i}, & E_{Na} &= \frac{1}{\phi} \ln \frac{Na_o}{Na_i}, \\ E_{Ks} &= \frac{1}{\phi} \ln \frac{K_o + p_{KNa} Na_o}{K_i + p_{KNa} Na_i}, & E_{Ca} &= \frac{1}{2\phi} \ln \frac{Ca_o}{Ca_i},\end{aligned}\tag{5}$$

258 and the individual currents are

$$\begin{aligned}I_{\text{Na}} &= G_{Na} m^3 h j (V - E_{Na}), & I_{\text{lo}} &= G_{to} r s (V - E_K), \\ I_{\text{Kr}} &= G_{Kr} \sqrt{K_o/5.4} x_{r1} x_{r2} (V - E_K), \\ I_{\text{Ks}} &= G_{Ks} x_s^2 (V - E_{Ks}), & I_{\text{K1}} &= G_{K1} x_{K1, \infty}(V) (V - E_K), \\ I_{\text{pK}} &= \frac{G_{pK} (V - E_K)}{1 + e^{(25-V)/5.98}}, & I_{\text{pCa}} &= \frac{G_{pCa} Ca_i}{K_{pCa} + Ca_i}, \\ I_{\text{bNa}} &= G_{bNa} (V - E_{Na}), & I_{\text{bCa}} &= G_{bCa} (V - E_{Ca}), \\ I_{\text{NaK}} &= \frac{P_{NaK} K_o Na_i}{(K_o + K_{mK})(Na_i + K_{mNa}) \Theta(V)}, \\ I_{\text{CaL}} &= G_{CaL} d f f_2 f_{cass} \Phi_{\text{GHK}}(V, Ca_{SS}), \\ I_{\text{NaCa}} &= \frac{k_{NaCa} (e^{\gamma V \phi} Na_i^3 Ca_o - \alpha e^{(\gamma-1)V \phi} Na_o^3 Ca_i)}{(K_{mNa}^3 + Na_o^3)(K_{mCa} + Ca_o)(1 + k_{sat} e^{(\gamma-1)V \phi})},\end{aligned}\tag{6}$$

where $\Theta(V) = 1 + 0.1245 e^{-0.1V\phi} + 0.0353 e^{-V\phi}$ and the L-type calcium driving term takes the Goldman–Hodgkin–Katz form

$$\Phi_{\text{GHK}}(V, C_{a_{SS}}) = \frac{4(V - 15) \phi F (0.25 C_{a_{SS}} e^{2(V-15)\phi} - C_{a_o})}{e^{2(V-15)\phi} - 1}. \quad (7)$$

Each of the twelve voltage-dependent gates relaxes toward a voltage-dependent steady state under Hodgkin–Huxley kinetics and is advanced by an exponential (Rush–Larsen) update, while the SR release gate R and the five ion concentrations (Na_i , K_i , Ca_i , $C_{a_{SS}}$, $C_{a_{SR}}$) evolve from these currents together with SR release, uptake, leak, transfer, and rapid calcium buffering. The complete gate rate functions, concentration balances, and standard TP06 parameter values are released with our code.

A.3 Coverage-aware Score formula

With N_{pde} PDEs (here $N_{\text{pde}} = 5$), method m scores on PDE pde

$$Q_{m,\text{pde}} = 100 \times \frac{\log e_{\text{worst},\text{pde}} - \log e_{m,\text{pde}}}{\log e_{\text{worst},\text{pde}} - \log e_{\text{best},\text{pde}}}, \quad (8)$$

where $e_{m,\text{pde}}$ is the test nRMSE of method m on PDE pde, and $e_{\text{best},\text{pde}} = \min_m e_{m,\text{pde}}$ and $e_{\text{worst},\text{pde}} = \max_m e_{m,\text{pde}}$ are the lowest and highest nRMSE achieved by any method on that PDE. The best method on a PDE scores 100, the worst 0, and a missing (“–”) result is assigned $Q_{m,\text{pde}} = 0$. The Score is the mean over all PDEs,

$$\text{Score}_m = \frac{1}{N_{\text{pde}}} \sum_{\text{pde}=1}^{N_{\text{pde}}} Q_{m,\text{pde}}, \quad (9)$$

where the method m ranges over the five compared frameworks (WEB-PDE-LLM, LLM-only, CodePDE, MCP-Sim, and PINNsAgent). The same construction applies to token cost (graded in log space like nRMSE, lower is better). Pass rate is not relative-graded: each cell’s pass-rate percentage enters directly (a missing result counting as 0), and $Q_{m,\text{pde}}$ is that percentage.

B Additional Results

This appendix reports the full test nRMSE table referenced from the *Accuracy* paragraph of the main text, the per-PDE pass-rate and token-cost measurements summarized in Figure 2 and in the *Reliability and cost* paragraph, together with qualitative field comparisons, the full ablation table, and the Solve time comparison.

B.1 Test nRMSE

B.2 Qualitative field comparisons

Figure 3 shows the predicted field itself, not just the scalar nRMSE, for the Gemini 3.5 Flash backbone on a single held-out initial condition. The layout is the same for all three PDEs: one column per PDE, the five methods stacked as blocks, and within each block the method’s final-time field next to its pointwise |error| against the reference on a log color scale, so near-perfect and badly diverged methods stay legible together. The Python baselines are re-run from their trained `best_solver.py` on this instance; WEB-PDE-LLM’s field is its saved browser output, and PINNsAgent’s is read from its network checkpoint and interpolated onto a regular grid where needed. Each block is annotated with its nRMSE and a wall-clock solve time. For WEB-PDE-LLM, which logs no per-instance timing, the time is its one recorded measurement rescaled to the test horizon, which is why it differs from ??.

B.3 Ablations

Dropping the parse-validation stage degrades accuracy on four of the five PDEs, most sharply on FK, where nRMSE rises from 2.80×10^{-4} to 7.77×10^{-3} , and it lowers the mean pass rate from 96.0% to 84.0%, confirming that explicitly checking the extracted Δt , boundary conditions, and parameter set catches unstable configurations the Code Agent would otherwise commit to. Additionally removing

Table 2: Test nRMSE by PDE type, method, and LLM backbone (lower is better). The best value in each PDE column is shaded, the second best underlined; “–” marks a case for which the method never produced a valid solver. The corresponding aggregated accuracy *Score* is reported in Figure 2.

Test nRMSE ($\downarrow$)	Advection	Burgers	Heat	FK	TNNP
WEB-PDE-LLM					
Gemini 3.5 Flash	2.56e-3	<u>1.46e-3</u>	3.89e-4	<u>2.81e-4</u>	<u>1.32e-5</u>
GPT-5.5	<u>1.70e-3</u>	7.24e-4	1.86e-6	2.80e-4	4.67e-6
Claude Opus 4.8	<u>1.88e-3</u>	<u>1.46e-3</u>	3.91e-4	9.57e-3	3.03e-3
DeepSeek V4-Pro	<u>1.70e-3</u>	<u>1.16e-3</u>	3.85e-5	6.95e-3	3.35e-6
LLM-only					
Gemini 3.5 Flash	1.36e-2	2.36e-2	2.12e-7	4.15e-4	–
GPT-5.5	1.36e-2	1.73e-2	2.12e-7	–	–
Claude Opus 4.8	1.32e-2	2.33e-2	2.12e-7	–	–
DeepSeek V4-Pro	1.32e-2	1.73e-2	2.13e-7	–	–
CodePDE					
Gemini 3.5 Flash	1.36e-2	2.36e-2	2.12e-7	4.15e-4	–
GPT-5.5	3.50e-2	1.74e-2	1.26e-6	3.12e-4	–
Claude Opus 4.8	1.32e-2	2.33e-2	1.27e-6	4.78e-3	–
DeepSeek V4-Pro	1.32e-2	1.72e-2	1.37e-6	2.79e-4	–
MCP-Sim					
Gemini 3.5 Flash	1.33e-2	3.75e-2	3.79e-4	–	–
GPT-5.5	1.36e-2	1.93e-2	1.79e-6	–	–
Claude Opus 4.8	1.36e-2	1.97e-2	3.79e-4	–	–
DeepSeek V4-Pro	9.59e-1	1.67e-2	7.81e-3	–	6.26e-1
PINNsAgent					
Gemini 3.5 Flash	1.89e-2	1.71e-1	1.24e-2	5.03e-1	–
GPT-5.5	1.01e-2	1.96e-1	7.85e-3	6.28e-1	–
Claude Opus 4.8	1.75e-2	2.41e-1	1.24e-2	7.27e-1	–
DeepSeek V4-Pro	2.72e-3	6.17e-1	8.69e-3	3.90e0	–

Table 3: Ablation on the GPT-5.5 backbone: effect of response parsing and autonomous debugging on test nRMSE. Lower is better.

Test nRMSE ($\downarrow$)	Advection	Burgers	Heat	FK	TNNP	Mean Pass Rate ($\uparrow$)
GPT-5.5						
WEB-PDE-LLM (full)	<u>1.70e-3</u>	<u>7.24e-4</u>	<u>6.61e-5</u>	<u>2.80e-4</u>	<u>4.67e-6</u>	96.0%
– Parse	2.56e-3	9.57e-4	1.45e-4	7.77e-3	3.82e-6	84.0%
– Parse – Debug	3.95e-2	1.33e-3	1.45e-4	7.77e-3	3.82e-6	80.0%

the Debug Agent leaves the FK and TNNP cells unchanged but costs another order of magnitude on advection ($2.56 \times 10^{-3} \rightarrow 3.95 \times 10^{-2}$) and a further 4 points of pass rate, indicating that shader repair mostly recovers cases that compile-fail or diverge outright rather than refining already-correct kernels. The one cell that does not follow the trend is TNNP, where the ablated variants report a marginally lower nRMSE (3.82×10^{-6} versus 4.67×10^{-6}); with a 12-point lower pass rate on the same configuration, this reflects averaging over a smaller and easier set of surviving runs rather than a genuine accuracy gain.

B.4 Pass rate

Accuracy is only meaningful on cases a framework can actually run, so Table 4 reports the fraction of grid scales for which the generated solver executes and produces output. WEB-PDE-LLM has by far the strongest TNNP pass rate, ranging from 50% (Gemini 3.5 Flash) to 100% (Claude Opus 4.8), and it stays at 70–90% on FK. CodePDE is highly reliable on the four simpler cases, at 100% almost everywhere, but never produces a running TNNP solver on any backbone, which is the failure the coverage-aware Score is designed to penalize. MCP-Sim degrades earliest, dropping to 40–80%

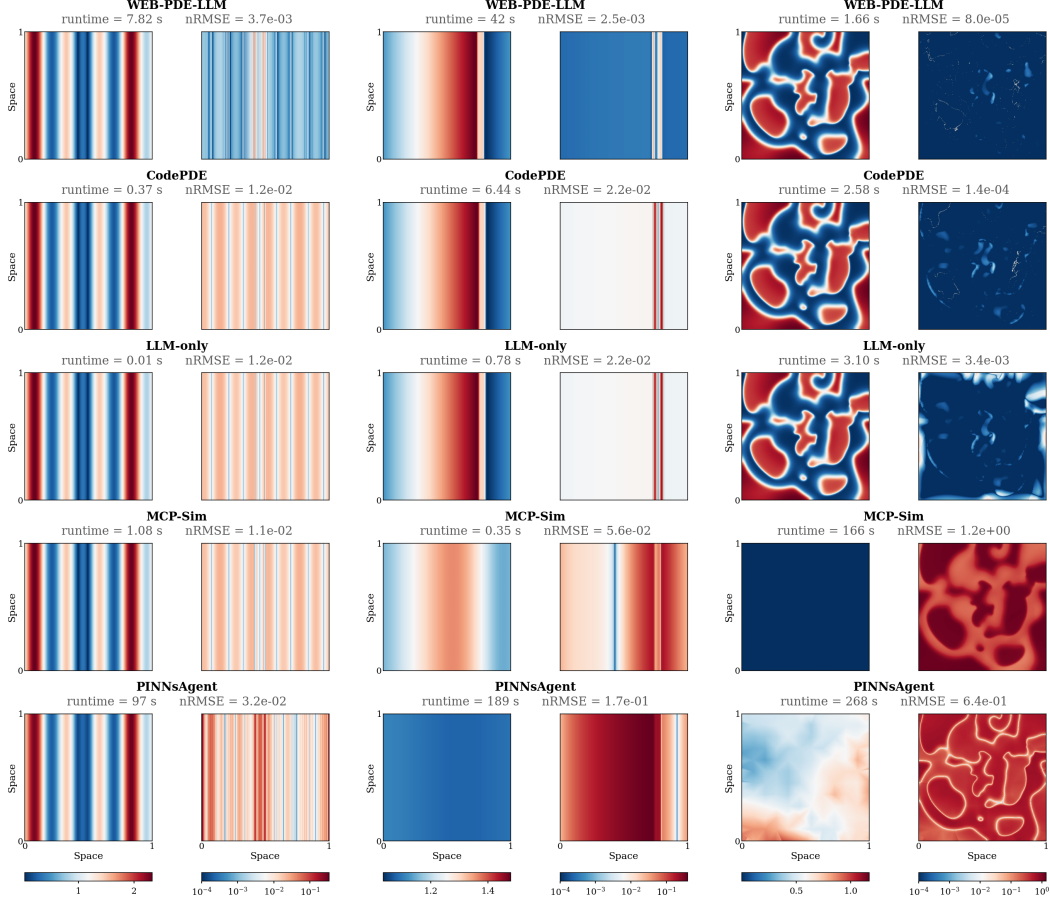


Figure 3: Qualitative field comparison, Gemini 3.5 Flash, one held-out initial condition; left to right, one column per PDE (Advection, Burgers', Fenton–Karma), one block per method. Each block shows its final-time field (left) and pointwise $|\text{error}|$ on a log scale (right), annotated with nRMSE and solve time. WEB-PDE-LLM and CodePDE track the reference closely on all three PDEs; LLM-only stays close at a fraction of the cost. MCP-Sim smooths out Burgers' shock and collapses to an all-zero Fenton–Karma field, and PINNsAgent misses the shock and the wavefront structure entirely.

312 already on Burgers and failing FK on every backbone; its lone TNNP success is a single passing
313 scale for DeepSeek V4-Pro (10%). PINNsAgent passes 100% nearly everywhere, but this measures
314 only that training completed: read together with Table 2, a completed PINNsAgent run still returns
315 errors of order 10^{-1} , so pass rate alone overstates its usefulness.

316 B.5 Token cost

317 Table 5 reports input/output tokens per PDE together with the total API cost of running each backbone
318 over the whole suite, including failed runs. The multi-agent loop is not free: WEB-PDE-LLM
319 consumes $2\text{--}4\times$ the tokens of one-shot LLM-only generation on the simple cases, because parse
320 validation, verification, and shader repair each issue additional calls. It nevertheless ends up cheaper
321 end-to-end than the other agentic baselines (\$1.39–\$13.76 per backbone against \$2.15–\$21.64 for
322 CodePDE and \$2.49–\$31.54 for MCP-Sim), since those systems burn roughly half a million tokens
323 per backbone on TNNP without ever producing a working solver. PINNsAgent is by far the cheapest
324 in tokens (\$0.26–\$1.82), as the LLM only proposes a training configuration; its cost is paid in GPU
325 training time instead, and Table 2 shows what that buys in accuracy. Backbone choice dominates the
326 absolute numbers: DeepSeek V4-Pro runs the full WEB-PDE-LLM suite for \$1.39 despite emitting
327 the most output tokens, a consequence of its per-token price rather than of shorter generations.

Table 4: Pass rate by PDE type, method, and LLM backbone, computed as successful scales divided by all scales. Higher is better.

Pass rate ($\uparrow$)	Advection	Burgers	Heat	FK	TNNP
WEB-PDE-LLM					
Gemini 3.5 Flash	100.0%	100.0%	100.0%	90.0%	50.0%
GPT-5.5	100.0%	100.0%	100.0%	90.0%	90.0%
Claude Opus 4.8	80.0%	100.0%	100.0%	80.0%	100.0%
DeepSeek V4-Pro	90.0%	90.0%	100.0%	70.0%	80.0%
LLM-only					
Gemini 3.5 Flash	100.0%	80.0%	100.0%	90.0%	0.0%
GPT-5.5	100.0%	100.0%	100.0%	0.0%	0.0%
Claude Opus 4.8	100.0%	30.0%	80.0%	0.0%	0.0%
DeepSeek V4-Pro	100.0%	60.0%	100.0%	0.0%	0.0%
CodePDE					
Gemini 3.5 Flash	100.0%	90.0%	100.0%	100.0%	0.0%
GPT-5.5	100.0%	100.0%	100.0%	100.0%	0.0%
Claude Opus 4.8	100.0%	100.0%	100.0%	100.0%	0.0%
DeepSeek V4-Pro	100.0%	100.0%	100.0%	80.0%	0.0%
MCP-Sim					
Gemini 3.5 Flash	100.0%	80.0%	90.0%	0.0%	0.0%
GPT-5.5	80.0%	50.0%	100.0%	0.0%	0.0%
Claude Opus 4.8	80.0%	80.0%	70.0%	0.0%	0.0%
DeepSeek V4-Pro	90.0%	40.0%	80.0%	0.0%	10.0%
PINNsAgent					
Gemini 3.5 Flash	100.0%	100.0%	100.0%	100.0%	0.0%
GPT-5.5	100.0%	100.0%	100.0%	100.0%	0.0%
Claude Opus 4.8	100.0%	100.0%	100.0%	100.0%	0.0%
DeepSeek V4-Pro	100.0%	100.0%	100.0%	100.0%	0.0%

C Agent Prompts

The exact prompt templates issued to each agent in Section 3’s parse/code/verify/debug loop, reproduced verbatim (Python `.format()` placeholders such as {PDEs} are filled in with the parsed metadata or raw PDE description at call time). The Debug and Code Agents share a `return_format` placeholder that requests the corrected/completed HTML verbatim, and `context_info` carries the browser console log, WebGL error, and/or the last-measured nRMSE. The ablation variants discussed in Section B.3 (“– Parse”, “– Parse – Debug”) use structurally identical Code/Debug prompts with the parsed-JSON fields (PDEs, bc, parameter_values) replaced by the raw PDE description text and the parse/validation and Debug Agent calls skipped entirely; they are omitted here as near-duplicates.

System Prompt (all agents)

You are an intelligent AI researcher for coding, numerical algorithms, and scientific computing.

Your goal is to conduct cutting-edge research in the field of PDE solving by leveraging and creatively improving existing algorithms to maximize performances based on feedbacks.

Follow the user’s requirements carefully and make sure you understand them.
Always use Markdown cells to present your code.

Table 5: Token cost by PDE type, method, and LLM backbone. Each entry is input/output tokens. The final column reports the estimated API cost in USD for that backbone across all PDE cases (including failed runs), using official token prices (gemini-3.5-flash \$1.50/\$9.00, GPT-5.5 \$2.50/\$15.00, claude-opus-4-8 \$5.00/\$25.00, deepseek-v4-pro \$0.435/\$0.87 per 1M in/out). PINNsAgent runs its LLM search once and reuses it, so its token counts and USD are scaled $\times 10$ to the 10-scale budget the other methods use; three DeepSeek runs that completed only 5 of 10 scales (LLM-only, CodePDE, and MCP-Sim on Heat) are likewise scaled $\times 2$. Lower is better.

Token cost ($\downarrow$)	Advection	Burgers	Heat	FK	TNNP	Cost (USD)
WEB-PDE-LLM						
Gemini 3.5 Flash	19.3k/10.6k	20.1k/13.5k	20.6k/13.0k	65.1k/38.2k	553.0k/304.6k	\$4.44
GPT-5.5	27.8k/13.0k	24.0k/14.2k	20.3k/10.6k	57.7k/33.8k	248.5k/184.3k	\$4.78
Claude Opus 4.8	39.9k/38.3k	52.0k/37.9k	31.6k/34.8k	103.0k/78.4k	323.3k/251.1k	\$13.76
DeepSeek V4-Pro	37.4k/141.7k	38.3k/201.4k	18.3k/69.9k	151.7k/308.7k	455.3k/520.0k	\$1.39
LLM-only						
Gemini 3.5 Flash	6.2k/7.2k	6.3k/13.4k	6.5k/10.7k	14.5k/18.8k	56.2k/73.5k	\$1.25
GPT-5.5	5.7k/7.2k	5.8k/9.7k	6.0k/34.8k	13.1k/14.4k	53.6k/60.5k	\$2.11
Claude Opus 4.8	8.5k/16.3k	8.7k/17.9k	9.1k/13.7k	19.0k/21.2k	78.0k/96.6k	\$4.76
DeepSeek V4-Pro	5.6k/33.1k	5.8k/83.3k	6.8k/388.6k	12.8k/98.2k	49.8k/235.2k	\$0.76
CodePDE						
Gemini 3.5 Flash	9.3k/14.1k	25.7k/39.9k	9.7k/15.9k	17.7k/24.1k	477.4k/513.3k	\$6.27
GPT-5.5	8.7k/19.7k	8.8k/22.8k	9.0k/78.0k	16.1k/26.7k	471.8k/513.8k	\$11.20
Claude Opus 4.8	13.4k/23.7k	13.6k/27.1k	14.0k/25.0k	23.9k/38.9k	553.1k/627.4k	\$21.64
DeepSeek V4-Pro	10.0k/39.1k	17.3k/93.9k	9.8k/344.7k	44.3k/206.4k	343.7k/1570.7k	\$2.15
MCP-Sim						
Gemini 3.5 Flash	52.0k/39.3k	66.6k/50.8k	58.8k/47.5k	184.7k/167.5k	543.1k/511.5k	\$8.71
GPT-5.5	76.4k/62.6k	72.0k/60.3k	66.2k/156.4k	175.9k/164.4k	565.1k/616.0k	\$18.29
Claude Opus 4.8	97.7k/75.9k	125.0k/103.7k	114.9k/96.5k	252.5k/232.0k	549.2k/525.7k	\$31.54
DeepSeek V4-Pro	47.6k/164.4k	76.5k/347.9k	60.7k/776.5k	136.2k/462.5k	390.7k/754.9k	\$2.49
PINNsAgent						
Gemini 3.5 Flash	39.0k/4.2k	39.2k/4.3k	40.9k/4.2k	38.7k/4.2k	38.7k/4.1k	\$0.48
GPT-5.5	33.9k/3.3k	34.0k/3.4k	35.4k/10.3k	33.6k/11.9k	33.6k/3.2k	\$0.91
Claude Opus 4.8	49.8k/4.5k	49.9k/4.6k	52.2k/4.6k	49.6k/4.5k	49.3k/4.4k	\$1.82
DeepSeek V4-Pro	33.5k/31.4k	33.5k/26.4k	37.3k/49.1k	35.5k/86.0k	33.2k/23.1k	\$0.26

Parse Agent

You are given a clarified simulation specification intended for WebGL.
Your task is to parse this paragraph into a structured JSON object that captures all key attributes needed for code generation.
Output only the JSON. No talk, no markdown.

```

### JSON Schema Requirements:
1. "PDEs": String. Full PDEs using LaTeX. IMPORTANT: Use double backslashes (e.g., \nabla) for all LaTeX commands to ensure valid JSON escaping. When there is a new line, use \n.
2. "number_of_state_variables": Integer.
3. "texture_size": Integer (e.g., 512).
4. "spatial_step": Float (the value of dx).
5. "domain_size": Float (the size of the spatial domain, e.g., 20 for [-10,10]).
6. "temporal_step": Float (the value of dt).
7. "time_horizon": Float (the total simulation time, e.g., 100.0).
8. "boundary_conditions": String or Object describing the conditions.
9. "parameter_values": Object mapping each parameter to its initial value.
10. "notes": String (optional, use null if no Important implementation notes)
### Example Output Format:
{
  "PDEs": "\frac{\partial u}{\partial t} = D \nabla^2 u \nabla",
  "number_of_state_variables": 3,
  "texture_size": 512,

```

```
"spatial_step": 0.01,
"domain_size": 20.0,
"temporal_step": 0.001,
"time_horizon": 100.0,
"boundary_conditions": "Periodic",
"parameter_values": {"D": 0.001, "C_m": 1.0, "tau_pv": 7.99},
}
```

Input:
{user_input}

Output:

340

Parsed-Content Checking Agent

Given this PDE description:
{pde_desc}

And this JSON parameter set:
{json}

Check if the parameters (especially temporal_step with CFL condition) are reasonable for this PDE.

Modify any values if needed for stability and accuracy.

Return the complete modified JSON (or the same JSON if no changes needed).

341

Code Agent

Given the {PDEs}, {coding_skeleton}, boundary condition {bc} and notes {notes}, return the complete glsl code implementation for the PDE solver in WebGL (do not put #version 300 es in your codes).

Output only the raw glsl code. No talk, no markdown, just code.

342

Debug Agent

The code {codes} is producing errors or high RMSE in this context: {context_info}, with the following pdes {PDEs}, boundary conditions {bc}, and parameter values:

{parameter_values}

{return_format}

343

Refine Agent (optional)

Refine the simulation code in {simulation_codes} to achieve a lower nRMSE than the current {nrmse} and maximize execution speed for the system described in {pde_desc}. Implement the most numerically accurate and computationally efficient integration and spatial discretization methods suitable for this specific system, optimizing GLSL performance and memory access patterns for peak efficiency. You must maintain the original application logic---specifically the CSV data loading, file downloading, and GUI infrastructure---entirely unchanged, focusing all improvements strictly on mathematical simulation quality and coding efficiency to ensure high-order precision and stability while preserving the existing structural integrity.

Output only the raw html code. No talk, no markdown, just code.

344